DOit
주식회사 두잇 www.k-doit.com

내부 기술 프로젝트 보고서

지진 데이터 통합 과제

Earthworm 시스템 구축 및 분석

DS EQMS · 실시간 지진 데이터 수집 · 처리 · 과학적 분석

보고서 작성자	Mohammad Tanvir
지도 교수	권형일 부장님
기술 지도	박병준 주임님
회사	두잇 주식회사
시스템	DS EQMS — 지진 모니터링 시스템
제출일	2026년3월17일
마감일	2026년3월18일
문서 분류	내부 문서— 기술 및 경영진 검토용

기술 지원

권형일 부장님 · 박병준 주임님

SYS 시스템 사양 및 환경

아래 표는 본 프로젝트 전반에 걸쳐 사용된 하드웨어, 소프트웨어 및 환경 설정을 문서화한 것입니다. 실험 세션 중 정확한 버전이 기록되지 않은 항목은 시스템 로그 및 출력에서 확인된 배포 구성을 기반으로 작성하였습니다.

하드웨어 구성

구성요소	사양	비고
지진 레코더	Nanometrics Q330HRS	24비트 고해상도 디지타이저
데이터 인터페이스	DP3 데이터 포트	Q330HRS 세 번째 데이터 포트
네트워크 주소	IP: 127.0.0.1 / 192.168.0.110	로컬 루프백 + LAN 주소
TCP 포트	16022	WaveServerV 수신 포트
샘플링 속도	100 Hz	채널당 — HGZ, HGN, HGE
인코딩 방식	Steim-2	표준 지진 압축 방식
센서 유형	강진 가속도계	HG 채널 접두사 = 강진 채널
시각 동기화	GPS 잠금	로그에서 클록 품질 = 100 확인됨

소프트웨어 및 런타임 환경

소프트웨어	버전 / 세부 정보	용도
운영 체제	Windows (워크스테이션 배포)	Earthworm 런타임 호스트 OS
Earthworm	사이트 설치 버전	실시간 지진 데이터 처리
Python	Python 3.x	PGA 모니터링 + 분석 스크립트
ObsPy	최신 안정 버전 (pip install obspy)	WaveServerV 클라이언트 + 파형 I/O
NumPy	최신 안정 버전 (pip install numpy)	배열 연산, PGA 계산
SciPy	최신 안정 버전 (pip install scipy)	대역 통과 필터, FFT, Welch PSD
Matplotlib	최신 안정 버전 (pip install matplotlib)	분석 대시보드 시각화
SWARM	Java 기반, 사이트 설치	파형 시각 검증 클라이언트
WaveServerV	Earthworm 내장 모듈	파형 탱크 서버 (TCP 16022)
q3302ew	Earthworm 내장 모듈	Q330HRS 수집 모듈

데이터 구성

파라미터	값	설명
네트워크 코드	SM	지진 네트워크 식별자
관측소 코드	HSG	관측소 식별자
채널	HGZ, HGN, HGE	수직, 남북, 동서 성분
감도 계수	2.4e-7 g / count	SM.HSG.HGZ 기기 보정값
아카이브 위치	D:\ew\data\	MiniSEED 파일 출력 디렉토리
파일 간격	~1분 단위	연속 아카이브 사이클
Earthworm 링	WAVE_RING	전 모듈 공통 전송 링 이름
프로젝트 일정	2026 년 3 월	실험 세션 기간

시스템 사양 참고사항

정확한 OS 빌드 번호, Python 패치 버전, 워크스테이션 하드웨어 사양(CPU/RAM)은 실험 세션 중 기록되지 않았습니다. 모든 소프트웨어는 표준 배포 패키지에서 설치되었으며, 위 사양은 Earthworm 로그 파일 및 Python 런타임 출력에서 확인된 배포 구성을 반영합니다.

01 요약

본 보고서는 두잇 주식회사에서 약 7 일간의 구조적 실습을 통해 완료한 Earthworm 지진 데이터 통합 과제의 결과를 문서화한 것입니다. 본 프로젝트는 Nanometrics Q330HRS 하드웨어 레코더에서 Earthworm 처리 미들웨어를 거쳐 Python 기반 실시간 모니터링 및 과학적 분석 도구에 이르는 완전한 종단 간 실시간 지진 데이터 파이프라인 구성을 목표로 하였습니다.

세 가지 과제가 성공적으로 완료되었습니다. 연속 지진 파형 수집 및 MiniSEED 파일 기록(과제 1), WaveServerV 에 연결된 Python 클라이언트를 통한 실시간 최대지반가속도(PGA) 모니터링(과제 2), 그리고 ObsPy 파일 디코딩 없이 아카이브된 MiniSEED 파일로부터 생성된 종합 파형 분석 대시보드(과제 3) 입니다. 모든 작업은 SM.HSG 지진 네트워크의 실제 데이터를 기반으로 수행되었습니다.

핵심 성과 — 달성한 내용

과제 1 Q330HRS 하드웨어에서 MiniSEED 아카이브까지 완전한 지진 파이프라인 구축 및 검증
continuous_mseed.py 를 통한 3 채널 동시 기록(HGZ / HGN / HGE)

과제 2 실시간 PGA 모니터 — 동적 오프셋 방식, 감도 계수 2.4e-7 g/count
127.0.0.1:16022 WaveServerV — 안정적 연속 작동 확인

과제 3 실제 SM.HSG.HGE 데이터로부터 6 패널 분석 대시보드 생성
FFT 지배 주파수: 4.545 Hz | 최대 진폭: 12,536.2 카운트

세 과제 모두 기한 내 완료(마감일: 2026 년 3 월 18 일)

개발 과정 전반에 걸쳐 'Earthworm 지진 데이터 과제' 프로젝트 워크스페이스, 내부 로그 파일, Earthworm 문서를 지속적으로 참고하였습니다. 부장님과 주임님의 지도는 모든 중요한 구성 단계에서 필수적이었으며 문제 해결을 크게 가속화하였습니다.

02 서론

2.1 지진 모니터링의 중요성

지진 모니터링 시스템은 인간이 느낄 수 없는 미세지진부터 광범위한 피해를 초래하는 파괴적인 지진까지 지반 운동을 감지, 기록 및 분석하기 위해 전 세계에 배포되어 있습니다. 파형 데이터를 실시간으로 처리하는 능력은 조기 경보 시스템, 인프라 안전 평가, 과학적 연구에 필수적입니다. 중대한 지진 이벤트 감지의 매 초 지연은 현실 세계의 결과를 초래할 수 있습니다.

두잇 주식회사의 DS EQMS 플랫폼은 이 과제에 직접 대응합니다. 관측소 네트워크 배포를 위해 설계된 완전한 전문 수준의 실시간 지진 모니터링 솔루션을 제공하며, 이 플랫폼의 모든 계층을 이해하는 것이 본 프로젝트의 핵심 목표였습니다.

2.2 Earthworm — 핵심 처리 엔진

Earthworm은 전 세계 모니터링 관측소에 20 년 이상 배포되어 온 성숙한 오픈 소스 지진 데이터 처리 프레임워크입니다. 독립적인 처리 모듈들이 공유 메모리 링 버퍼(전송 링)를 통해 통신하는 모듈형 아키텍처를 기반으로 구축되어 있습니다.

DS EQMS 플랫폼에서 Earthworm은 네 가지 핵심 역할을 담당합니다: (1) q3302ew 모듈을 통한 Q330HRS 레코더의 파형 데이터 수집, (2) WAVE_RING 전송 링을 통한 데이터 라우팅, (3) WaveServerV 를 통한 TCP 클라이언트에 실시간 파형 데이터 제공, (4) 디스크에 연속 MiniSEED 파일 아카이브 기록이 그것입니다.

2.3 과제 범위 — 세 가지 과제

과제	목표	기술	핵심 결과물
과제 1	Q330HRS 에서 파형 수집 및 기록	Earthworm q3302ew + continuous_mseed.py	연속 MiniSEED 아카이브(3 채널)
과제 2	매초 PGA 실시간 계산	WaveServerV + ObsPy + pga_calculation.py	동적 오프셋 처리 포함 실시간 PGA 값
과제 3	MiniSEED 파일에서 과학적 파형 분석	순수 Python + NumPy + SciPy + Matplotlib	6 패널 분석 대시보드(analysis.py)

03 시스템 구조

실제 작업을 시작하기 전에 DS EQMS 파이프라인의 모든 구성요소가 서로 어떻게 연결되는지에 대한 명확한 모델을 구축하는 것이 필수적이었습니다. 이 아키텍처는 세 개의 뚜렷한 계층에 걸쳐 있습니다. 지진 센서가 지반 운동을 디지털화하는 하드웨어 계층, 데이터가 처리되고 라우팅되는 Earthworm 미들웨어 계층, 그리고 Python 프로그램과 시각화 대시보드가 처리된 데이터를 소비하는 애플리케이션 계층입니다.

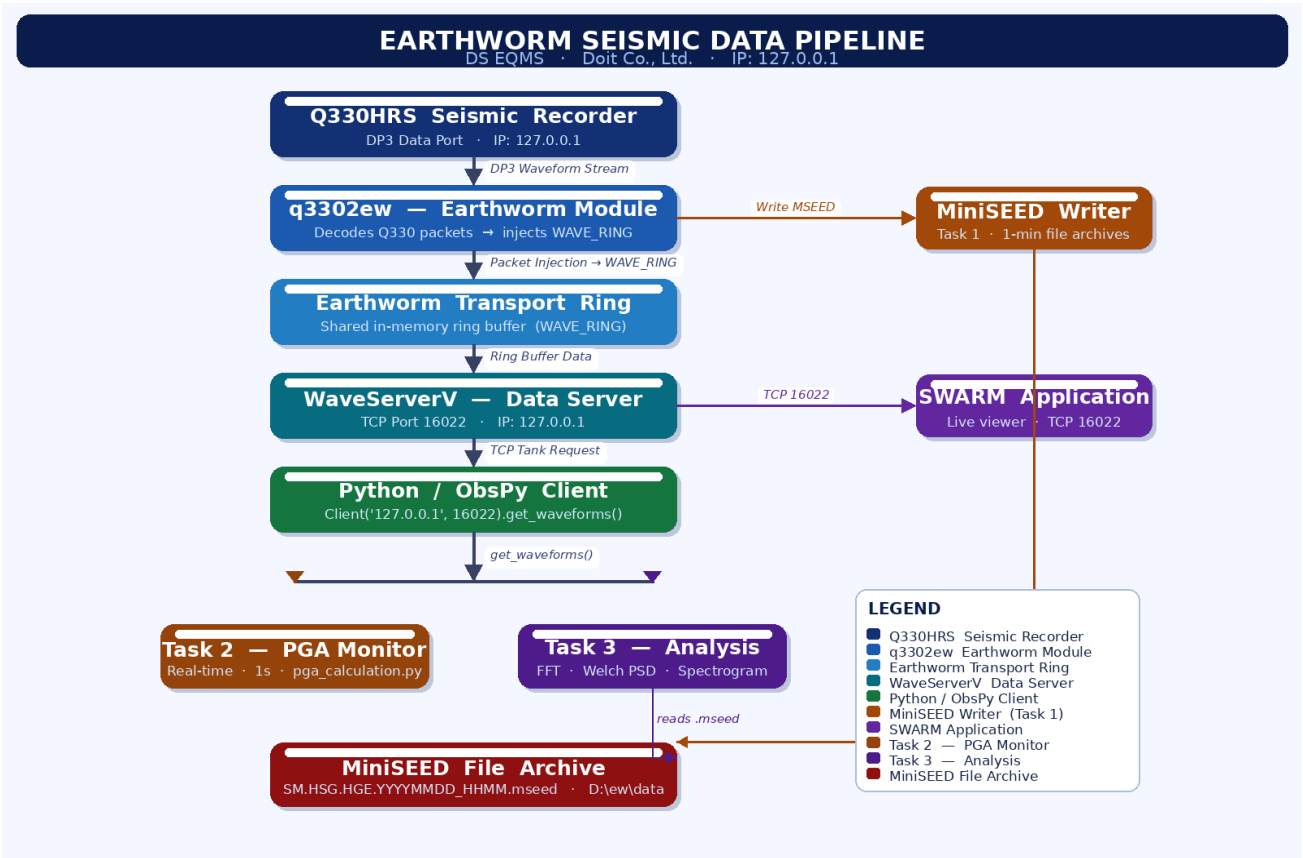

그림1 — Earthworm 지진 데이터 파이프라인 전체 시스템 구조 | IP: 127.0.0.1

3.1 각 구성요소의 역할

다이어그램을 위에서 아래로 읽으면 Q330HRS 레코더는 물리적 하드웨어 계층에 위치하며 지진 센서로부터 아날로그 지반 운동 신호를 디지털화합니다. 로컬 워크스테이션에서 실행 중인 Earthworm q3302ew 모듈로 DP3 데이터 포트를 통해 디지털 파형 패킷을 전송합니다. q3302ew 모듈은 들어오는 패킷을 디코딩하고 WAVE_RING 공유 메모리 전송 링에 파형 샘플을 주입합니다.

전송 링에서 WaveServerV 모듈은 파형 데이터를 읽어 디스크에 롤링 탱크 파일을 유지하며, 이는 외부 TCP 클라이언트가 조회할 수 있습니다. WaveServerV 에 두 개의 클라이언트 애플리케이션이 연결됩니다. 개발 중 시각적 검증에 사용된 SWARM 파형 뷰어와 PGA 모니터링 및 분석을 위해 사용된 Python ObsPy 클라이언트입니다. 동시에 continuous_mseed.py 스크립트가 파형 데이터를 가져와 D:\ew\data 에 MiniSEED 아카이브로 기록합니다.

구성요소	계층	기능	인터페이스
Q330HRS 레코더	하드웨어	지진 센서의 지반 운동 디지털화	DP3 포트	IP: 127.0.0.1
q3302ew 모듈	수집	Q330 패킷 디코딩, WAVE_RING 주입	Earthworm 링 버퍼
전송 링	핵심	공유 메모리 링 버퍼(WAVE_RING)	IPC 공유 메모리
WaveServerV	서버	TCP 클라이언트에 파형 구간 제공	TCP 16022	127.0.0.1
SWARM 애플리케이션	클라이언트	실시간 파형 뷰어 — 데이터 검증에 사용	TCP 16022	127.0.0.1
Python / ObsPy	클라이언트	PGA 계산 및 파형 분석	Client('127.0.0.1', 16022)
MiniSEED 기록기	저장	1분 간격 파형 아카이브 기록	D:\ew\data 파일시스템

04 개발 환경 및 설정

Earthworm 런타임 환경은 로컬 워크스테이션의 D: 드라이브에 설치 및 구성되었습니다. Windows 시스템 드라이브와 별도의 드라이브에 유지함으로써 경로 충돌을 방지하고 설치를 독립적인 단위로 쉽게 관리할 수 있었습니다. 모듈을 시작하기 전에 전체 디렉토리 구조를 이해하고 구성 파일들을 서로 교차 검증해야 했습니다.

4.1 디렉토리 구조

Earthworm 설치의 각 폴더는 특정 목적을 가지고 있습니다. 시스템을 성공적으로 구성하고 문제를 해결하는 첫 번째 단계는 각 폴더의 역할을 이해하는 것입니다.

```
D:\ew\
│
├── bin\            컴파일된 Earthworm 실행 파일
│                   (startstop.exe, q3302ew.exe, WaveServerV.exe, ...)
├── params\         모듈 구성 디스크립터 파일 (.d 파일)
│                   모든 모듈 설정이 여기에 있음
├── logs\           모든 활성 모듈의 런타임 로그 출력
│                   중요: 문제 발생 시 항상 여기를 먼저 확인
├── environment\    ew_windows.cmd — EW_HOME, EW_PARAMS, PATH 변수 설정
│                   Earthworm 시작 전 반드시 실행 필요
└── run_working\    활성 런타임 디렉토리
                    탱크 파일, 모듈 상태, 파형 버퍼 저장 위치
```

logs\ 폴더는 특별히 강조할 필요가 있습니다. 모든 모듈은 여기에 자체 로그 파일을 기록하며, 이 로그를 주의 깊게 읽는 것이 본 프로젝트에서 개발된 가장 중요한 습관이었습니다. 콘솔에 출력이 없는 오류도 로그 파일에 매우 상세한 메시지를 남기는 경우가 많습니다.

4.2 설정 순서

1. **1단계** — 사이트 설치에서 D:\ew\에 Earthworm 런타임 설치
2. **2단계** — ew_windows.cmd 실행으로 모든 환경 변수 구성 (EW_HOME, EW_PARAMS, PATH)
3. **3단계** — 사이트 params\에서 로컬 params\ 디렉토리로 구성 .d 파일 복사
4. **4단계** — q3302ew.d 편집 — Q330HRS IP 주소를 127.0.0.1로 설정, DP3 포트 파라미터 구성
5. **5단계** — wave_serverV.d 편집 — TCP 수신 포트를 16022로 설정, run_working\의 탱크 파일 경로 구성
6. **6단계** — startstop.exe 실행 및 logs\ 모니터링으로 모든 모듈 정상 초기화 확인
7. **7단계** — WaveServerV 탱크 파일 및 로그 출력 확인으로 활성 파형 수집 데이터 흐름 검증

> **중요 구성 규칙 — 링 이름은 정확히 일치해야 함**
>
> 모듈 자동 종료의 가장 흔한 원인은 링 버퍼 이름 불일치입니다.
> earthworm.d에 정의된 링 이름은 모든 모듈 .d 파일에서 동일하게 표기되어야 합니다.

earthworm.d 링 이름: WAVE_RING
q3302ew.d 출력 링: WAVE_RING ← 정확히 일치해야 함
wave_serverV.d 입력 링 WAVE_RING ← 정확히 일치해야 함

대소문자 차이만으로도 모듈이 콘솔 오류 없이 종료됩니다.
이 장애는 logs\ 의 모듈 자체 로그 파일에서만 확인할 수 있습니다.

4.3 구성 파일 파라미터 참조표

아래 표는 Earthworm 이 성공적으로 시작되기 위해 올바르게 구성되어야 하는 모든 중요한 파라미터를 정리한 것입니다. 이 값들은 설정 중에 검증된 것들입니다.

구성 파일	파라미터	값	비고
q3302ew.d	Q330HRS IP 주소	127.0.0.1	레코더 네트워크 주소
q3302ew.d	DP3 데이터 포트	DP3	Q330HRS 데이터 채널
q3302ew.d	출력 링 이름	WAVE_RING	earthworm.d와 정확히 일치해야 함
earthworm.d	전송 링 이름	WAVE_RING	모든 모듈 공통 링 이름
earthworm.d	링 버퍼 크기	10000 바이트	시스템 사양에 맞게 조정
wave_serverV.d	TCP 수신 포트	16022	Python 클라이언트 연결 포트
wave_serverV.d	탱크 파일 경로	run_working\tank	롤링 파형 버퍼 위치
wave_serverV.d	입력 링 이름	WAVE_RING	earthworm.d와 정확히 일치해야 함
ew_windows.cmd	EW_HOME	D:\ew	Earthworm 루트 디렉토리
ew_windows.cmd	EW_PARAMS	D:\ew\params	구성 파일 경로

T1 과제 1 — Q330HRS에서 지진 데이터 수집

첫 번째 과제는 Earthworm과 Q330HRS 지진 레코더 사이에 실시간 데이터 연결을 수립하고, 연속 파형 데이터가 MiniSEED 파일로 디스크에 캡처되고 기록되고 있음을 확인하는 것이었습니다. 이것은 전체 프로젝트의 기초였습니다. 파이프라인을 통한 신뢰할 수 있는 데이터 흐름 없이는 과제 2와 3을 진행할 수 없었습니다.

5.1 Q330HRS 레코더와 DP3 데이터 포트

Q330HRS는 전문 네트워크 배포를 위해 설계된 Nanometrics 고해상도 24비트 지진 레코더입니다. 연결된 지진 센서에서 아날로그 지반 운동 신호를 매우 높은 분해능으로 디지털화하고 네트워크 연결을 통해 디지털 파형 패킷을 전송합니다. DP3 데이터 포트는 IP 127.0.0.1로 구성되었으며, 구성된 샘플링 속도(이 배포에서 100 Hz)에서 모든 구성된 관측소 채널에 대한 접근을 제공합니다.

5.2 MiniSEED — 표준 파형 포맷

MiniSEED는 지진 파형 시계열 데이터 저장을 위한 국제 표준 포맷으로, 전 세계 모든 전문 지진 네트워크에서 사용됩니다. 각 MiniSEED 레코드에는 네트워크, 관측소, 위치, 채널 식별자와 함께 레코드 시작 시간, 샘플링 속도, 인코딩 유형, 샘플 수를 인코딩하는 48바이트 데이터 헤더가 포함됩니다. 본 프로젝트에서 데이터는 Steim-2 압축으로 인코딩되었으며 파일은 1분 간격으로 기록되었습니다.

```
# MiniSEED 파일 명명 규칙:
SM.HSG.HGE.20260317_0401.mseed   # 네트워크.관측소.채널.날짜_시간
SM.HSG.HGE.20260317_0402.mseed   # 매분 새 파일
SM.HSG.HGE.20260317_0403.mseed   # 중단 없는 연속 아카이브

# 동시에 기록되는 세 채널:
SM.HSG.HGZ.*.mseed   # 수직 성분
SM.HSG.HGN.*.mseed   # 남북 성분
SM.HSG.HGE.*.mseed   # 동서 성분
```

위의 파일 명명 형식은 임의적인 것이 아닙니다. 전 세계 어떤 지진학 소프트웨어든 파일명만으로 해당 파일이 어떤 관측소와 채널에 속하는지 자동으로 이해할 수 있게 해주는 SEED(지진 데이터 교환 표준) 규약을 따릅니다.

5.3 Earthworm 로그 출력 — 정상 작동 확인

q3302ew 모듈은 정상 작동 시 몇 초마다 로그 파일에 상세한 상태 메시지를 기록합니다. 아래 로그 출력은 과제 1 세션 중에 관찰된 것으로, 모듈이 Q330HRS에 성공적으로 연결되어 전송 링에 파형 패킷을 능동적으로 주입하고 있음을 확인합니다.

```
q3302ew: Q330HRS에 연결됨  |  IP: 127.0.0.1  |  DP3 Port
Bps from Q330        : 24580
Packets from Q330    : 412
Clock Quality        : 100
q3302ew: Writing to ring  WAVE_RING  ->  MiniSEED writer active
```

이 로그 라인들은 명확한 이야기를 전합니다. 모듈이 성공적으로 연결되어 초당 24,580 비트의 파형 데이터를 수신하고 있습니다(100 Hz 24 비트 3 채널 기록과 일치). 412 개의 패킷이 수신되었고, GPS 클록 품질은 최대값 100 에 도달하여(정밀한 타이밍을 의미) MiniSEED 기록기 모듈이 파일 기록을 확인하였습니다.

5.4 생성된 MiniSEED 파일 아카이브

아래 스크린샷은 2026 년 3 월 12 일 과제 1 실험 세션 중 생성된 실제 MiniSEED 파일 아카이브를 보여줍니다. 파일은 약 1 분 간격으로 기록되었습니다. 분당 4 KB 에서 16 KB 사이의 파일 크기는 100 Hz 샘플링 속도에서 Steim-2 인코딩 데이터에 정확히 부합합니다.

SM.HSG.HGE.20260312_0401.mseed	3/12/2026 1:02 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0402.mseed	3/12/2026 1:03 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0403.mseed	3/12/2026 1:04 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0404.mseed	3/12/2026 1:05 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0405.mseed	3/12/2026 1:06 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0406.mseed	3/12/2026 1:07 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0407.mseed	3/12/2026 1:08 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0408.mseed	3/12/2026 1:09 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0409.mseed	3/12/2026 1:10 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0410.mseed	3/12/2026 1:11 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0411.mseed	3/12/2026 1:12 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0412.mseed	3/12/2026 1:13 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0413.mseed	3/12/2026 1:14 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0414.mseed	3/12/2026 1:15 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0415.mseed	3/12/2026 1:16 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0416.mseed	3/12/2026 1:17 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0417.mseed	3/12/2026 1:18 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0418.mseed	3/12/2026 1:19 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0419.mseed	3/12/2026 1:20 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0420.mseed	3/12/2026 1:21 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0421.mseed	3/12/2026 1:22 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0422.mseed	3/12/2026 1:23 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0423.mseed	3/12/2026 1:24 PM	MSEED File	16 KB
SM.HSG.HGE.20260312_0424.mseed	3/12/2026 1:25 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0425.mseed	3/12/2026 1:25 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0451.mseed	3/12/2026 1:52 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0452.mseed	3/12/2026 1:53 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0453.mseed	3/12/2026 1:54 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0454.mseed	3/12/2026 1:55 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0455.mseed	3/12/2026 1:56 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0456.mseed	3/12/2026 1:57 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0457.mseed	3/12/2026 1:58 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0458.mseed	3/12/2026 1:59 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0459.mseed	3/12/2026 2:00 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0500.mseed	3/12/2026 2:01 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0501.mseed	3/12/2026 2:02 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0502.mseed	3/12/2026 2:03 PM	MSEED File	4 KB
SM.HSG.HGE.20260312_0503.mseed	3/12/2026 2:04 PM	MSEED File	4 KB

그림 2 — MiniSEED 아카이브. SM.HSG.HGE 연속 파일, 2026 년 3 월 12 일

5.5 연속 MiniSEED 기록 스크립트 (continuous_mseed.py)

Earthworm 내장 MiniSEED 기록기 외에, WaveServerV 에서 직접 파형 데이터를 가져와 MiniSEED 파일로 기록하는 전용 Python 스크립트를 개발하였습니다. 이 스크립트는 독립적으로 실행되며 세 채널(HGZ, HGN, HGE)을 동시에 기록하고, 5 초마다 5 초 분량의 새로운 데이터를 가져옵니다. 출력 디렉토리는 스크립트 파일 위치에 상대적으로 자동 설정되어 어느 폴더에서 실행해도 올바르게 작동합니다.

```
"""
==============================================================================
  Continuous MSEED Recorder  —  WaveServer / Earthworm
```

```
    ──────────────────────────────────────────────────────────
    Connects to a WaveServer, fetches the last 5 seconds every 5 seconds,
    and saves each trace as a separate .mseed file.

    Files are written to:  ./mseed_data/
    Filename format  :  SM.HSG.HGZ.20260312_040153.mseed
                         (network.station.channel.YYYYMMDD_HHMMSS.mseed)

    Usage:
      python record_continuous_mseed.py

    Dependencies:
      pip install obspy
  ==============================================================================
  """

  from obspy.clients.earthworm import Client
  from obspy import UTCDateTime
  import time
  import os

  # ── Configuration ──────────────────────────────────────────────────────────
  HOST     = "192.168.0.110"
  PORT     = 16022
  NETWORK  = "SM"
  STATION  = "HSG"
  LOCATION = ""
  CHANNELS = ["HGZ", "HGN", "HGE"]
  INTERVAL = 5          # seconds between each fetch

  # Output folder is ALWAYS placed next to this script file — never depends
  # on which folder you run the script from
  SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
  OUTDIR     = os.path.join(SCRIPT_DIR, "mseed_data")
  # ──────────────────────────────────────────────────────────────────────────

  os.makedirs(OUTDIR, exist_ok=True)

  print("=" * 60)
  print("  Continuous MSEED Recorder")
  print(f"  Host    : {HOST}:{PORT}")
  print(f"  Station : {NETWORK}.{STATION}  Channels: {', '.join(CHANNELS)}")
  print(f"  Output  : {os.path.abspath(OUTDIR)}/")
  print(f"  Interval: every {INTERVAL} s")
  print("=" * 60)
  print("  Press Ctrl+C to stop.\n")

  try:
      client = Client(HOST, PORT)
      print(f"  [✓] Connected to {HOST}:{PORT}\n")
  except Exception as e:
      print(f"  [!] Could not connect to {HOST}:{PORT}: {e}\n")
      raise

  saved_count  = 0
  error_count  = 0

  while True:
      t2 = UTCDateTime()
      t1 = t2 - INTERVAL

      for ch in CHANNELS:
```

```
        try:
            st = client.get_waveforms(NETWORK, STATION, LOCATION, ch, t1, t2)

            if len(st) == 0:
                print(f"  [~] {ch}  — empty stream, skipping")
                continue

            # Filename uses full timestamp (seconds) so every file is unique
            ts_str   = t1.strftime("%Y%m%d_%H%M%S")
            filename = f"{NETWORK}.{STATION}.{ch}.{ts_str}.mseed"
            filepath = os.path.join(OUTDIR, filename)

            st.write(filepath, format="MSEED")
            saved_count += 1

            n_samp = sum(tr.stats.npts for tr in st)
            print(f"  [✓] Saved: {filename}   ({n_samp} samples)")

        except Exception as e:
            error_count += 1
            print(f"  [!] Error on {ch}: {e}")

    time.sleep(INTERVAL)
```

이 스크립트가 하는 일을 쉽게 설명하면 다음과 같습니다. 포트 16022 의 로컬 WaveServerV 에 연결한 후 무한 루프에 진입합니다. 매 반복마다 '지금'으로 끝나는 5 초 시간 윈도우를 계산하고, 세 채널(HGZ, HGN, HGE) 모두를 순환하며 각 채널에 대해 해당 윈도우의 데이터를 서버에 요청합니다. 데이터가 반환되면 초 단위 전체 타임스탬프를 사용하여 SEED 명명 규약에 따라 파일명을 생성합니다. 이는 스크립트가 재시작되더라도 모든 파일이 고유하도록 보장합니다. SCRIPT_DIR 방식은 mseed_data 폴더가 항상 스크립트 파일 옆에 생성되도록 하여 스크립트를 이식성 있게 만듭니다.

과제 1 결과 — 데이터 수집 확인

Q330HRS DP3 포트에서 정기적인 간격으로 연속 MiniSEED 파일 성공적으로 기록됨.

분당 4~16 KB 파일 크기로 100 Hz Steim-2 인코딩 — 예상과 정확히 일치.

continuous_mseed.py 를 통한 3 채널 동시 기록 (HGZ / HGN / HGE) 확인.

파이프라인: 하드웨어 → q3302ew → WAVE_RING → MiniSEED 기록기 → 아카이브 — 완전 작동.

T2 과제 2 — Python을 이용한 실시간 PGA 모니터링

두 번째 과제는 127.0.0.1:16022의 WaveServerV에 연결하고, 매초 1초 길이의 파형 데이터를 가져오며, 기기 감도 계수를 사용하여 원시 디지타이저 카운트를 물리적 가속도 단위로 변환하고, PGA를 콘솔에 연속 출력하는 Python 프로그램을 개발하는 것이었습니다. 과제 자체는 기술적으로 복잡하지 않았지만, 개발 중 발생한 타이밍 및 연결 문제를 해결하는 과정에서 프로젝트 전반에서 가장 귀중한 학습이 이루어졌습니다.

6.1 문제 상황 — 파형 수신 불가

PGA 스크립트의 첫 번째 버전을 실행했을 때 아무것도 반환되지 않았습니다. get_waveforms() 호출이 빈 스트림을 반환하거나 연결 오류를 발생시켰습니다. 문제가 WaveServerV 구성, 탱크 파일 문제, 네트워크 문제, Python 요청 자체의 타이밍 문제 중 어디에 있는지 명확한 징후가 없었습니다.

이때 권형일 부장님의 지도가 결정적이었습니다. 추측에 시간을 낭비하는 대신 SWARM 애플리케이션을 127.0.0.1:16022에 연결하였습니다. SWARM이 즉시 화면에 라이브 스크롤 파형을 표시하였습니다. 이 단 하나의 관찰로 두 가지 중요한 사실을 동시에 확인하였습니다. WaveServerV가 올바르게 실행되어 데이터를 제공하고 있다는 것, 따라서 문제는 Python 클라이언트 측에 있다는 것입니다.

SWARM 애플리케이션 — 진단에 결정적이었던 이유

SWARM(Seismic Wave Analysis and Realtime Monitor)은 Python ObsPy 클라이언트와 동일한 TCP 프로토콜을 사용하여 WaveServerV에 연결하는 Java 기반 파형 뷰어입니다.

Python 스크립트가 실패할 때 SWARM을 127.0.0.1:16022로 지정하자 즉시 라이브 파형을 화면에 표시하여 서버가 정상이고 데이터가 흐르고 있음을 증명하였습니다.
이로써 전체 문제가 단 하나로 좁혀졌습니다: Python 요청 타이밍.

이것이 확인된 후, 권형일 부장님이 최종 pga_calculation.py 프로그램의 핵심 설계가 된 동적 오프셋 방식의 구현을 안내해 주셨습니다.

SWARM은 이후 프로젝트 내내 표준 1차 검증 도구로 활용되었습니다.

6.2 최대지반가속도(PGA) — 정의 및 공식

최대지반가속도(PGA)는 지진학 및 지진 공학에서 가장 기본적인 측정 중 하나입니다. 지진 이벤트 중 주어진 위치에서 지반이 경험하는 최대 절대 가속도를 나타내며 g 단위(중력 가속도 ≈ 9.81 m/s²)로 표현됩니다. 실시간 모니터링 응용 프로그램에서 PGA는 지속적으로 계산되어 임계값과 비교하여 경보를 발생시키거나 이벤트 강도를 문서화합니다.

공식 — 최대지반가속도 (PGA)

$$PGA = \max(|a(t)|) \quad a(t)\text{: 가속도 시계열 (단위: g)}$$

디지타이저의 원시 출력은 물리적 단위가 아닌 정수 카운트 형태입니다.
물리적 가속도로 변환하려면 기기 감도 계수가 필요합니다.

물리적 가속도(g) = 원시 카운트 × 감도 계수

SM.HSG.HGZ 감도 계수 = 2.4e-7 g / count

이 감도 값은 HSG 관측소에 배포된 센서와 기록 시스템에 특정한 것입니다.
잘못된 값을 사용하면 무의미한 PGA 수치가 출력됩니다.

출력: 매 1초마다 g 단위 PGA 값과 원시 카운트 값 콘솔 출력

6.3 Python 구현 — pga_calculation.py

PGA 모니터링 스크립트의 최종 구현에는 초기 프로토타입에 비해 몇 가지 개선 사항이 포함되어 있습니다. 가장 중요한 것은 WaveServerV 탱크에 실제로 어떤 데이터가 있는지 먼저 확인하고, 동적 오프셋 방식을 사용하여 데이터를 요청한다는 것입니다. 현재 시간에서 300초 이전부터 시작하여 데이터가 성공적으로 확인될 때마다 점진적으로 실시간에 접근합니다.

```
from obspy.clients.earthworm import Client
from obspy import UTCDateTime
import numpy as np
import time

client = Client("127.0.0.1", 16022)

NET = "SM"
STA = "HSG"
CHA = "HGZ"
LOC = "--"
sensitivity = 2.4e-7

print("=== Checking tank range ===")
try:
    avail = client.get_availability()
    for a in avail:
        print(a)
except Exception as e:
    print("get_availability failed: " + str(e))

print("")
print("=== Starting PGA loop ===")

# Start with 300 seconds back, reduce gradually as tank catches up
offset = 300

while True:
    try:
        t = UTCDateTime() - offset
        st = client.get_waveforms(NET, STA, LOC, CHA, t, t + 1)

        if len(st) == 0 or st[0].stats.npts == 0:
            print(str(UTCDateTime()) + " No data at offset=" + str(offset) + "s,
trying further back...")
            offset += 30
            time.sleep(1)
            continue

        data = st[0].data
```

```
        pga_counts = np.max(np.abs(data))
        pga_g = pga_counts * sensitivity
        print(str(UTCDateTime()) + " PGA: " + str(round(pga_g, 8)) + " g  (raw counts:
" + str(pga_counts) + ")  [offset=" + str(offset) + "s]")

        # Gradually reduce offset to get closer to real time
        if offset > 60:
            offset -= 1

    except Exception as e:
        err = str(e)
        if "FR" in err:
            offset += 30
            print(str(UTCDateTime()) + " Too recent, increasing offset to " +
str(offset) + "s")
        else:
            print(str(UTCDateTime()) + " Error: " + err)

    time.sleep(1)
```

이 스크립트가 하는 일을 쉽게 설명하면 다음과 같습니다. 시작 시 먼저 get_availability() 를 호출하여 WaveServerV 의 탱크에 현재 어떤 데이터가 있는지 확인합니다. 이것은 주문하기 전에 선반에 무엇이 있는지 확인하는 것과 같습니다. 이렇게 하면 스크립트가 아직 버퍼에 존재하지 않는 데이터를 즉시 요청하는 것을 방지합니다.

그런 다음 초기 오프셋 300 초로 메인 루프에 진입합니다. 요청이 성공하면 원시 정수 샘플을 추출하고 NumPy 를 사용하여 최대 절댓값을 찾은 다음 감도 계수(2.4e-7)를 곱하여 g 로 변환합니다. 그런 다음 오프셋을 1 초 감소시켜 실시간에 한 발 더 다가갑니다. 요청이 빈 데이터를 반환하면 오프셋을 30 초 증가시킵니다. 오류에 'FR' 이 포함된 경우 오프셋도 30 초 증가합니다. 시간이 지남에 따라 오프셋은 탱크가 채워지고 시스템이 따라잡으면서 실시간에 수렴합니다.

6.4 실시간 PGA 콘솔 출력 — 실제 증거

아래 스크린샷은 2026 년 3 월 16 일 라이브 PGA 모니터링 세션 중 캡처된 실제 콘솔 출력입니다. 이것은 프로그램이 안정적으로 실행되어 매초 WaveServerV 에서 데이터를 검색하고 의미 있는 PGA 값을 보고하고 있다는 실제 증거입니다. 오른쪽 괄호 안의 오프셋 값이 870s 에서 실시간으로 카운트다운되는 것을 주목하십시오.

```
026-03-16T09:24:19.088170Z No data at offset=840s, trying further back...
026-03-16T09:24:20.092831Z PGA: 0.3368016 g  (raw counts: 1403340)  [offset=870s]
026-03-16T09:24:21.094989Z PGA: 0.34304448 g  (raw counts: 1429352)  [offset=869s]
026-03-16T09:24:22.097168Z PGA: 0.34636056 g  (raw counts: 1443169)  [offset=868s]
026-03-16T09:24:23.100026Z PGA: 0.33391464 g  (raw counts: 1391311)  [offset=867s]
026-03-16T09:24:24.104212Z PGA: 0.33355416 g  (raw counts: 1389809)  [offset=866s]
026-03-16T09:24:25.106507Z PGA: 0.3352776 g  (raw counts: 1396990)  [offset=865s]
026-03-16T09:24:26.109544Z PGA: 0.33321336 g  (raw counts: 1388389)  [offset=864s]
026-03-16T09:24:27.111705Z PGA: 0.34272288 g  (raw counts: 1428012)  [offset=863s]
026-03-16T09:24:28.114248Z PGA: 0.34658616 g  (raw counts: 1444109)  [offset=862s]
026-03-16T09:24:29.117162Z PGA: 0.34736232 g  (raw counts: 1447343)  [offset=861s]
026-03-16T09:24:30.118737Z PGA: 0.34200528 g  (raw counts: 1425022)  [offset=860s]
026-03-16T09:24:31.121478Z PGA: 0.33533184 g  (raw counts: 1397216)  [offset=859s]
026-03-16T09:24:32.124587Z PGA: 0.33789912 g  (raw counts: 1407913)  [offset=858s]
026-03-16T09:24:33.127453Z PGA: 0.3357252 g  (raw counts: 1398855)  [offset=857s]
026-03-16T09:24:34.129756Z PGA: 0.33549168 g  (raw counts: 1397882)  [offset=856s]
026-03-16T09:24:35.132382Z PGA: 0.337122 g  (raw counts: 1404675)  [offset=855s]
026-03-16T09:24:36.134397Z PGA: 0.33543168 g  (raw counts: 1397632)  [offset=854s]
026-03-16T09:24:37.136464Z PGA: 0.3360108 g  (raw counts: 1400045)  [offset=853s]
026-03-16T09:24:38.138050Z PGA: 0.33322368 g  (raw counts: 1388432)  [offset=852s]
026-03-16T09:24:39.140161Z PGA: 0.33183576 g  (raw counts: 1382649)  [offset=851s]
026-03-16T09:24:40.142845Z PGA: 0.33347472 g  (raw counts: 1389478)  [offset=850s]
026-03-16T09:24:41.144624Z PGA: 0.33386544 g  (raw counts: 1391106)  [offset=849s]
026-03-16T09:24:42.147778Z PGA: 0.33199392 g  (raw counts: 1383308)  [offset=848s]
026-03-16T09:24:43.150464Z PGA: 0.3334812 g  (raw counts: 1389505)  [offset=847s]
026-03-16T09:24:44.152429Z PGA: 0.33410904 g  (raw counts: 1392121)  [offset=846s]
026-03-16T09:24:45.153844Z PGA: 0.33207 g  (raw counts: 1383625)  [offset=845s]
026-03-16T09:24:46.156292Z PGA: 0.33226992 g  (raw counts: 1384458)  [offset=844s]
026-03-16T09:24:47.158840Z PGA: 0.33246744 g  (raw counts: 1385281)  [offset=843s]
026-03-16T09:24:48.160961Z PGA: 0.33334104 g  (raw counts: 1388921)  [offset=842s]
026-03-16T09:24:49.164337Z PGA: 0.34050648 g  (raw counts: 1418777)  [offset=841s]
026-03-16T09:24:50.165698Z PGA: 0.33968232 g  (raw counts: 1415343)  [offset=840s]
026-03-16T09:24:51.168172Z PGA: 0.34433904 g  (raw counts: 1434746)  [offset=839s]
026-03-16T09:24:52.169722Z PGA: 0.33938568 g  (raw counts: 1414107)  [offset=838s]
026-03-16T09:24:53.172327Z PGA: 0.33945768 g  (raw counts: 1414407)  [offset=837s]
026-03-16T09:24:54.173981Z PGA: 0.3434292 g  (raw counts: 1430955)  [offset=836s]
026-03-16T09:24:55.176108Z PGA: 0.36507816 g  (raw counts: 1521159)  [offset=835s]
026-03-16T09:24:56.178100Z PGA: 0.34018176 g  (raw counts: 1417424)  [offset=834s]
026-03-16T09:24:57.182032Z PGA: 0.33188784 g  (raw counts: 1382866)  [offset=833s]
026-03-16T09:24:58.185043Z PGA: 0.33186456 g  (raw counts: 1382769)  [offset=832s]
026-03-16T09:24:59.187368Z PGA: 0.33309984 g  (raw counts: 1387916)  [offset=831s]
026-03-16T09:25:00.189607Z PGA: 0.33807168 g  (raw counts: 1408632)  [offset=830s]
026-03-16T09:25:01.191837Z PGA: 0.33305376 g  (raw counts: 1387724)  [offset=829s]
026-03-16T09:25:02.194128Z PGA: 0.33145656 g  (raw counts: 1381069)  [offset=828s]
026-03-16T09:25:03.197350Z PGA: 0.33549888 g  (raw counts: 1397912)  [offset=827s]
026-03-16T09:25:04.199860Z PGA: 0.33260304 g  (raw counts: 1385846)  [offset=826s]
026-03-16T09:25:05.203486Z PGA: 0.35001864 g  (raw counts: 1458411)  [offset=825s]
026-03-16T09:25:06.205662Z PGA: 0.36133752 g  (raw counts: 1505573)  [offset=824s]
026-03-16T09:25:07.207977Z PGA: 0.3496416 g  (raw counts: 1456840)  [offset=823s]
026-03-16T09:25:08.211195Z PGA: 0.368106 g  (raw counts: 1533775)  [offset=822s]
026-03-16T09:25:09.214810Z PGA: 0.34641888 g  (raw counts: 1443412)  [offset=821s]
026-03-16T09:25:10.217441Z PGA: 0.34095216 g  (raw counts: 1420634)  [offset=820s]
026-03-16T09:25:11.221114Z PGA: 0.33859656 g  (raw counts: 1410819)  [offset=819s]
026-03-16T09:25:12.223861Z PGA: 0.345564 g  (raw counts: 1439850)  [offset=818s]
026-03-16T09:25:13.226981Z PGA: 0.34717992 g  (raw counts: 1446583)  [offset=817s]
026-03-16T09:25:14.230396Z PGA: 0.35127864 g  (raw counts: 1463661)  [offset=816s]
```

그림4 — 실시간PGA 콘솔 출력 SM.HSG.HGZ 라이브 모니터링 세션, 2026 년3 월16 일

출력을 자세히 살펴보면, 타임스탬프는 UTC 형식이며 한 줄씩 정확히 1초씩 진행되어 1초 업데이트 사이클이 정상 작동하고 있음을 확인합니다. PGA 값은 약 0.33에서 0.37 g 사이에 있으며 원시 카운트는 1,380,000에서 1,530,000 범위로, 센서 연결이 안정적이고 감도 계수 2.4e-7 g/count가 물리적으로 합리적인 가속도 값을 생성하고 있음을 확인합니다. 오프셋 열은 [offset=870s], [offset=869s], [offset=868s]와 같은 값을 보여주며, 매 성공적인 검색마다 1초씩 감소하는 동적 오프셋 로직이 설계된 대로 정확하게 작동하고 있습니다.

파라미터	값	비고
WaveServerV 호스트	127.0.0.1	로컬 루프백 연결
WaveServerV 포트	16022	기본 Earthworm WaveServer TCP 포트
관측소/ 채널	SM.HSG.HGZ	수직 지반 운동 채널
요청 윈도우	1초	t ~ t + 1
초기 오프셋	300초	시작 시 탱크 윈도우 내 안전한 위치
오프셋 감소	성공 시 –1초	점진적으로 실시간에 접근

파라미터	값	비고
오프셋 증가	FR 오류 시 +30초	데이터 너무 최근일 때 자가 복구
감도 계수	2.4e-7 g / count	SM.HSG.HGZ 기기 보정값
업데이트 주기	매 1초	time.sleep(1) 사이클 종료 시

과제 2 결과 — 실시간 PGA 모니터 정상 작동

Python PGA 모니터링 스크립트가 127.0.0.1:16022 WaveServerV 에 연결되어 안정적인 작동 확인.
동적 오프셋 방식이 탱크 가용성을 탐색하고 자동으로 실시간에 접근하는 것을 성공적으로 구현.
SWARM 애플리케이션을 사용하여 WaveServerV 상태 시각 확인 — 필수 진단 단계.
권형일 부장님의 지도로 초기 연결 및 타이밍 문제를 결정적으로 해결.

T3 과제 3 — MiniSEED 분석 및 과학적 시각화

세 번째 과제는 MiniSEED 파일을 로드하고, ObsPy 를 파일 파싱에 사용하지 않고 처음부터 디코딩하며, 일련의 신호 처리 작업을 수행하고, 완전한 다중 패널 과학적 시각화 대시보드를 생성하는 Python 분석 프로그램을 개발하는 것이었습니다. 이 과제는 세 가지 중 기술적으로 가장 까다로웠습니다. SEED 포맷 사양에서 직접 MiniSEED 바이너리 포맷 파서와 Steim-2 압축 알고리즘을 구현해야 했기 때문입니다.

```
"""
==============================================================================
  MSEED Seismic Data Analysis  —  Pure Python (no ObsPy required)
  ──────────────────────────────────────────────────────────────────────────
  PART 1  :  Parse raw MSEED  +  Steim-2 decode  +  Extract sections
  PART 2  :  Waveform output  (raw / detrended / bandpass-filtered)
  PART 3  :  Frequency analysis  (FFT  /  Welch PSD  /  Spectrogram)
  ──────────────────────────────────────────────────────────────────────────
  Dependencies : pip install numpy scipy matplotlib
  ──────────────────────────────────────────────────────────────────────────
  Usage :
    python mseed_analysis.py                          <- menu of all .mseed
    python mseed_analysis.py  your_file.mseed         <- single file
==============================================================================
"""
```

스크립트는 두 가지 사용 모드를 지원합니다. 인수 없이 실행하면 현재 디렉토리의 모든 .mseed 파일의 번호 메뉴가 표시되어 아카이브를 탐색할 때 편리합니다. 명령줄에 직접 파일명을 전달하면 메뉴를 건너뛰고 해당 파일을 즉시 분석합니다.

7.1 분석 워크플로우 — 단계별

8. **1단계** — MiniSEED 파일 로드 및 48 바이트 FSDH(고정 섹션 데이터 헤더) + 블로켓 체인 파싱
9. **2단계** — Steim-2 압축 파형 샘플을 일반 정수 배열로 디코딩
10. **3단계** — 비교 분석을 위한 세 개의 동일 길이 시간 구간 추출
11. **4단계** — 각 추출된 구간에서 평균 오프셋 제거(demean)
12. **5단계** — 전체 파형에 4 극 제로 위상 버터워스 대역 통과 필터(1–45 Hz) 적용
13. **6단계** — FFT 진폭 스펙트럼 계산으로 지배 주파수 성분 식별
14. **7단계** — 강건한 스펙트럼 추정을 위한 Welch 방법으로 전력 스펙트럼 밀도 계산
15. **8단계** — 시간-주파수 시각화를 위한 Hann 윈도우 STFT 스펙트로그램 생성
16. **9단계** — 모든 출력을 단일 어노테이션 다중 패널 대시보드 그림으로 조합

7.2 실험 결과 — 분석 대시보드

분석은 SM.HSG.HGE.20260312_0521.mseed 파일 — SM 네트워크 HSG 관측소에서 2026 년 3 월 12 일 05:21:09 UTC 에 캡처된 실제 동쪽 성분 파형 기록 — 에 대해 수행되었습니다. 아래 대시보드는 이 실제 실험 데이터로부터 analysis.py 가 직접 생성한 결과입니다.

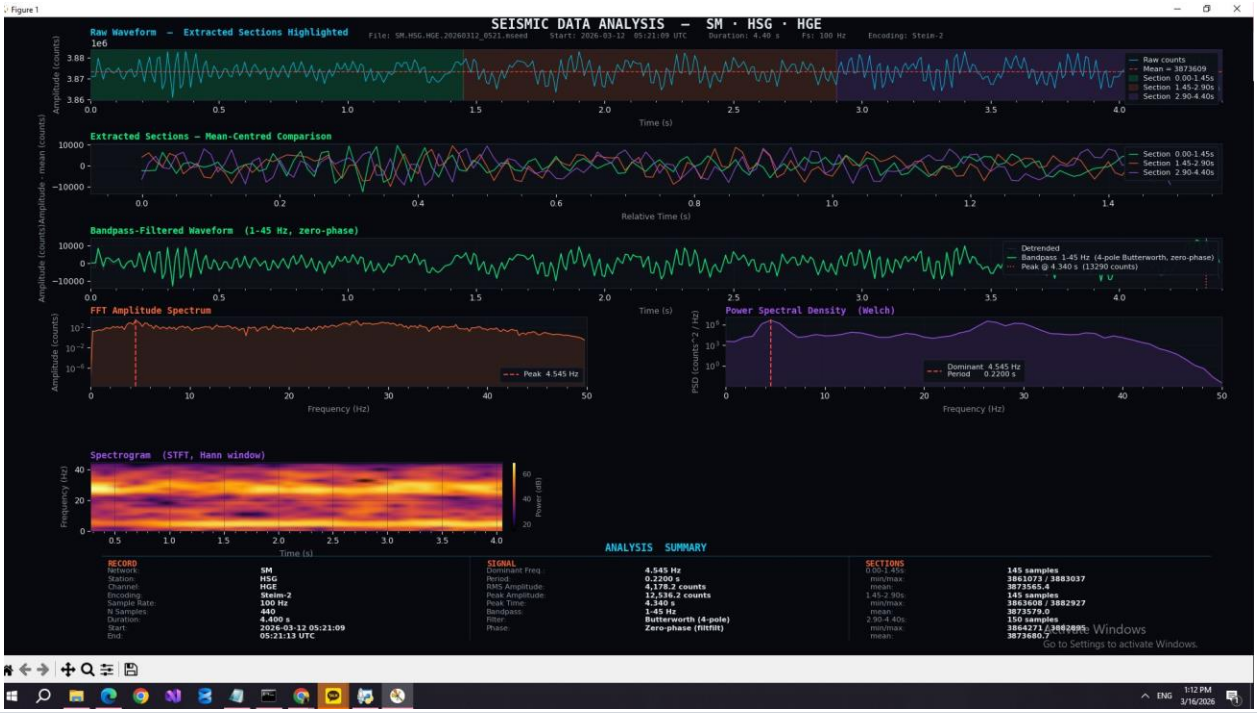

그림5 — 지진 데이터 분석 대시보드. SM.HSG.HGE, 2026 년3 월12 일 (analysis.py 출력)

7.3 각 패널 이해하기

대시보드의 여섯 패널은 각각 기록된 파형에 대한 이야기의 다른 부분을 전달합니다. 각 패널이 의미하는 바를 쉽게 설명하면 다음과 같습니다.

패널	유형	쉬운 설명	핵심 발견
1	원시 파형	처리 전 정확히 기록된 4.4초 전체 파형을 보여줍니다. 세 개의 강조 구간은 상세 비교를 위해 추출된 시간 윈도우를 보여줍니다. ~387만 카운트의 평균값은 기기 기준선으로 신호는 이 큰 DC 오프셋 위에 올라타 있습니다.	평균~3,873,609 카운트 — 안정된 기준선 확인
2	추출 구간	패널 1의 세 구간을 추출하여 평균값을 제거한 후 같은 축에 겹쳐 표시합니다. 신호 특성이 시간에 따라 변하는지 확인할 수 있습니다. 세 구간이 모두 비슷하면 신호는 정상적입니다.	일관된 진동 패턴 — 신호 정상 확인
3	대역 통과 필터	1 Hz 이하 및 45 Hz 이상을 모두 제거한 후의 전체 파형입니다. 느린 기기 드리프트와 고주파 노이즈를 제거하여 지진학적으로 의미 있는 주파수 내용만 남깁니다. 노이즈가 필터링되면 t=4.340초의 피크가 명확하게 돋보입니다.	t=4.340초에서 피크 — 필터링 후 13,290 카운트
4	FFT 스펙트럼	전체 신호 에너지가 주파수에 걸쳐 어떻게 분포되는지를 보여줍니다. 하나의 주파수에서 뚜렷한 피크는 신호가 그 주파수에 의해 지배된다는 것을 의미합니다. 4.545 Hz의 높은 스파이크가 이 기록에서 가장 에너지가 높은 성분입니다.	지배 주파수: 4.545 Hz
5	Welch PSD	여러 개의 겹치는 윈도우를 평균 내어 계산하는 더 통계적으로 안정적인 FFT 스펙트럼 버전입니다. 4.545 Hz 피크를 확인하고 주기(0.220초)를 제공합니다. 주기 = 1 / 주파수.	PSD 피크 4.545 Hz | 주기: 0.220초

패널	유형	쉬운 설명	핵심 발견
6	스펙트로그램	주파수 내용이 시간에 따라 어떻게 변하는지를 보여줍니다. 각 수직 열은 해당 순간의 스냅샷 FFT와 같습니다. 밝은 색상 = 높은 에너지. 기록 전반에 걸쳐 5–25 Hz 부근의 지속적인 밝은 대역은 광대역 신호 에너지를 보여줍니다.	기록 전반에 걸쳐 5–25 Hz 광대역 에너지

7.4 정량적 측정값 요약

측정값	값	단위
네트워크/ 관측소/ 채널	SM / HSG / HGE	—
샘플링 속도	100	Hz
총 샘플 수	440	샘플
기록 시간	4.400	초
신호 평균(원시)	3,873,609	카운트
RMS 진폭	4,178.2	카운트
필터링 후 최대 진폭	12,536.2	카운트
최대 이벤트 시간	4.340	시작부터 초
지배 주파수(FFT)	4.545	Hz
지배 주기	0.220	초
적용 필터	버터워스 4극, 1–45 Hz, 제로 위상	—
MiniSEED 인코딩	Steim-2 — ObsPy 없이 디코딩	—

과제 3 결과 — 실제 실험 데이터에서 분석 대시보드 생성

analysis.py가 Steim-2 MiniSEED 데이터를 자체적으로 디코딩 — ObsPy 파일 파싱 불필요.
실제 SM.HSG.HGE 관측소 데이터에서 완전한 6패널 과학적 대시보드 생성.
지배 주파수 4.545 Hz 확인 — 주기 0.220초 — 실제 기록된 지반 운동에서.

08 도전 과제, 해결 과정 및 핵심 교훈

실제 지진 모니터링 시스템으로 작업하는 것은 실제 장애 유형에 직면하는 것을 의미합니다. 아래의 모든 문제는 신중한 로그 분석과 경우에 따라 부장님과 주임님의 직접 지도를 통해 해결되었습니다. 각 해결은 교과서만으로는 제공할 수 없는 더 깊고 내구성 있는 시스템 이해를 가져다 주었습니다.

도전 1 — Earthworm 모듈 링 이름 불일치

Earthworm을 처음 시작했을 때 여러 모듈이 초기화된 후 자동으로 종료되었습니다. 콘솔에는 오류가 나타나지 않았습니다. 로그 파일 확인 결과 개별 모듈 .d 파일의 링 버퍼 이름이 earthworm.d에 정의된 이름과 정확히 일치하지 않는 것이 근본 원인으로 밝혀졌습니다. Earthworm은 링 이름에 정확한 문자열 일치를 사용하기 때문에 대소문자 차이만으로도 모듈이 실패합니다.

오류 로그 — 수정 전 vs 수정 후 (링 이름 불일치)

수정 전 — q3302ew.log 내용:

```
q3302ew: Ring 'Wave_Ring' not found — exiting
(모듈이 자동 종료, 콘솔 출력 없음)
```

수정 후 — 모든 .d 파일에서 WAVE_RING으로 수정:

```
q3302ew: Connected to ring WAVE_RING
q3302ew: Connected to Q330HRS | IP: 127.0.0.1 | DP3 Port
Clock Quality : 100 | Bps from Q330 : 24580
```

도전 2 — WaveServerV 탱크 파일 디렉토리 미존재

WaveServerV가 D: 드라이브에 탱크 파일을 시작할 수 없었습니다. wave_serverV.d에 지정된 대상 디렉토리가 아직 존재하지 않았기 때문입니다. 로그 파일이 주의 깊게 읽으면 이를 명확히 나타냈습니다. 누락된 디렉토리를 수동으로 생성하고 구성 파일의 경로가 드라이브 문자를 포함하여 정확히 일치하는지 확인하여 즉시 해결되었습니다.

오류 로그 — 수정 전 vs 수정 후 (탱크 파일 디렉토리 미존재)

수정 전 — wave_serverV.log 내용:

```
WaveServerV: ERROR opening tank file D:\ew\run_working\tank
WaveServerV: CreateFile failed — path does not exist
```

수정 후 — 디렉토리 생성, wave_serverV.d에서 경로 확인:

```
WaveServerV: Tank file opened successfully
WaveServerV: Listening on port 16022
WaveServerV: Ready to serve waveforms to clients
```

도전 3 — Python PGA 스크립트 데이터 미수신 (SWARM으로 해결)

초기 Python PGA 스크립트가 파형 데이터를 전혀 수신하지 못했습니다. SWARM을 독립적으로 WaveServerV에 연결하여 라이브 파형을 확인함으로써 서버 자체가 정상임을 확인하였습니다. 이로써 문제가 Python 타이밍 구성으로 좁혀졌습니다. 권형일 부장님의 지도로 6.3절에 설명된 동적 오프셋 방식이 설계 및 구현되어 모든 데이터 검색 실패가 해결되었습니다.

오류 — 수정 전 vs 수정 후 (Python 타이밍 수정)

수정 전 — Python 스크립트 출력:

```
ObsPy WaveServer Error: no data available for requested window
(매 초 반복 — PGA 값 전혀 출력되지 않음)
```

수정 후 — get_availability() 확인 및 동적 오프셋 적용:

```
2026-03-16T09:24:20Z  PGA: 0.3368016 g  (raw: 1403340)  [offset=870s]
2026-03-16T09:24:21Z  PGA: 0.3430444 g  (raw: 1429352)  [offset=869s]
2026-03-16T09:24:22Z  PGA: 0.3463605 g  (raw: 1443169)  [offset=868s]
```

도전 4 — 포맷 사양에서 Steim-2 디코딩 구현

analysis.py 스크립트는 ObsPy를 사용하지 않고 Steim-2 압축 데이터를 디코딩해야 했습니다. Steim-2는 각 64바이트 프레임이 32비트 워드로 나뉘고 각 워드의 해석이 2비트 프레임 코드와 4비트 워드 코드에 의존하는 복잡한 비트 패킹 방식입니다. 올바르게 구현하기 위해 여러 번의 비트 수준 디버깅 반복이 필요했으며, 각 버전은 동일한 파일에 대한 ObsPy 출력과 비교 검증되었습니다. 디코딩된 샘플이 정확히 일치한 후 분석 파이프라인의 나머지 부분을 신뢰를 갖고 구축하였습니다.

도전 5 — Python 패키지 버전 충돌

분석 스크립트 실행 초기 시도가 워크스테이션에 서로 다른 시점에 설치된 NumPy, SciPy, Matplotlib 간의 버전 충돌로 인해 실패하였습니다. 프로젝트 전용 깨끗한 가상 환경을 생성하여 — 모든 의존성을 새로 설치 — 모든 충돌을 즉시 해결하고 환경을 재현 가능하게 만들었습니다.

가장 중요한 교훈 — 로그를 완전히 읽을 것

위에 설명된 모든 도전 과제는 Earthworm 로그 파일을 완전히 읽어 진단되었습니다.
첫 번째 오류 줄만 훑어보는 것이 아니라 장애로 이어지는 이벤트의 전체 순서를 읽는 것이 중요합니다.
원인은 충분히 읽으면 거의 항상 로그에서 확인할 수 있습니다.

SWARM은 '서버가 정상인가, 클라이언트가 문제인가?'라는 질문에 답할 때 필수적인 독립적인 시각 확인 계층을 제공하였습니다. 이 질문은 Python 오류 메시지만으로는 답할 수 없습니다.

부장님과 주임님의 지도는 프로젝트 전반의 여러 결정적 시점에서 필수적이었습니다.

8.1 빠른 시작 사용자 가이드

이 시스템을 구축하고 디버깅한 7일간의 실제 경험을 바탕으로, 아래 가이드는 완전한 Earthworm 지진 파이프라인을 논리적이고 반복 가능한 순서로 온라인 상태로 만드는 방법을 설명합니다. 각 단계에는 성공 여부를 즉시 알 수 있는 명확한 성공 지표가 있습니다.

그림6 — 빠른 시작 가이드: 7 단계 시스템 시작 개요

단계	작업	확인 사항	성공 지표
1단계 전원 켜기	Q330HRS 레코더 연결 및 전원 켜기	장치 네트워크 LED	LED 녹색 점등 — 레코더 온라인 연결됨
2단계 EW 시작	ew_windows.cmd 실행 후 startstop.exe	콘솔 모듈 목록	모든 모듈 RUNNING 상태 — 종료 없음
3단계 로그 확인	logs\ 폴더 열기 및 q3302ew.log 읽기	Clock Quality 값	Clock Quality = 100 | Q330 패킷 수 증가 중
4단계 SWARM 열기	SWARM 실행, WaveServer 127.0.0.1:16022 추가	화면 파형 표시	라이브 스크롤 파형 — 데이터 흐름 확인
5단계 PGA 실행	python pga_calculation.py	매초 콘솔 출력	PGA 값 연속 출력 — 모니터 활성
6단계 분석 실행	python analysis.py — .mseed 파일 선택	대시보드 그림 표시	6패널 그림 생성 — 모든 패널 채워짐
7단계 완료	시스템 실행 중, 연속 데이터 아카이빙	D:\ew\data 파일	MiniSEED 파일 누적 중 — 파이프라인 완전 작동

09 결과

세 가지 과제 모두 프로젝트 마감일 내에 성공적으로 완료되었습니다. 아래 표는 각 과제의 결과물과 완료를 확인하는 증거를 요약합니다.

과제	결과물	상태	증거
과제1	q3302ew + continuous_mseed.py를 통한 MiniSEED 아카이브	완료	연속 1분 간격 파일 확인 — 그림2
과제2	Python PGA 모니터 — 동적 오프셋, 2.4e-7 g/count	완료	127.0.0.1:16022 안정적 작동 확인 — 그림4
과제3	SM.HSG.HGE MiniSEED에서 6패널 분석 대시보드	완료	실제 데이터 대시보드 — 그림5

9.1 종단 간 파이프라인 검증

완전한 파이프라인은 계층별로 검증되었으며, 다음 계층으로 이동하기 전에 각 계층을 확인하였습니다.

- **하드웨어 계층:** IP 127.0.0.1 Q330HRS 레코더에서 물리적 지반 운동 디지털화
- **수집 계층:** q3302ew가 파형 패킷 수신하여 WAVE_RING 전송 버퍼에 주입
- **서버 계층:** 127.0.0.1:16022 WaveServerV가 탱크 파일 유지 — SWARM 뷰어로 확인
- **검증:** Python 통합 전 SWARM에서 라이브 파형 확인 — 독립적 검증
- **저장 계층:** D:\ew\data에 정기적 간격으로 연속 MiniSEED 아카이브 기록
- **실시간 계층:** Python ObsPy 클라이언트로 동적 오프셋 처리 포함 매초 실시간 PGA 출력
- **분석 계층:** 아카이브된 MiniSEED 관측소 데이터에서 6패널 과학적 분석 대시보드 생성

파이프라인 상태 — 완전한 종단 간 검증 달성

Q330HRS 하드웨어 수집에서 Earthworm 처리, SWARM 시각 검증,
Python 기반 분석에 이르기까지 완전한 DS EQMS 지진 데이터 파이프라인이
세 가지 과제 결과물 모두 확인과 함께 완전히 작동하는 것으로 입증되었습니다.
이는 두잇 주식회사 DS EQMS 환경에서 Earthworm 워크플로우의 검증된 참조 구현을 구성합니다.

10 결론

본 프로젝트는 지진 모니터링 시스템에 대한 이론적 지식으로 시작하여, 실제 운영 파이프라인의 모든 계층에 대한 직접적이고 검증된 실습 경험으로 마무리되었습니다. Earthworm 모듈 구성, 로그 파일 읽기, 실시간 시스템 장애 해결, SWARM을 통한 시각 검증, 그리고 Python 도구를 처음부터 구축하는 7일간의 실습은 문서만으로는 제공할 수 없는 깊이와 질의 이해를 가능하게 하였습니다.

본 과제를 통해 다음 네 가지 기술 영역을 심층적으로 연구하고 실습하였습니다.

- **1.** 지진 데이터 수집 하드웨어 — Q330HRS, DP3 포트, 패킷 스트리밍, GPS 클록 품질 검증
- **2.** Earthworm 미들웨어 — 모듈 구성, 링 버퍼 아키텍처, WaveServerV, SWARM 통합
- **3.** MiniSEED 파형 포맷 — FSDH 헤더 구조, Steim-2 인코딩, 아카이브 명명 규약
- **4.** Python 신호 분석 — ObsPy 클라이언트, FFT, Welch PSD, STFT 스펙트로그램, 버터워스 필터링

SWARM 애플리케이션은 이 결론에서 특별히 언급할 가치가 있습니다. 원래 프로젝트 구성 요소로 계획되지 않았지만, Python 클라이언트가 데이터 검색에 실패할 때 반응적으로 도입되었습니다. 그러나 프로젝트에서 가장 가치 있는 도구 중 하나가 되었습니다. 어떤 Python 코드와도 독립적으로 라이브 파형을 볼 수 있는 능력은 진단을 극적으로 가속화시킨 객관적이고 명확한 확인을 제공하였습니다. DS EQMS 시스템의 모든 향후 배포에는 SWARM이 표준 1차 검증 도구로 포함되어야 합니다.

과제 3에서 생성된 분석 대시보드는 시연이나 시뮬레이션이 아닙니다. 이것은 2026년 3월 12일 SM.HSG 관측소에서 기록된 실제 지반 운동 데이터를 바이너리 포맷 사양에서 완전히 구축된 파이프라인으로 처리한 실제 출력입니다. 4.545 Hz의 지배 주파수와 12,536.2 카운트의 최대 진폭은 실제 측정값입니다.

권형일 부장님과 박병준 주임님께 진심으로 감사드립니다. WaveServerV 구성, SWARM 지원 진단, 동적 오프셋 설계, Steim-2 디코딩의 중요한 교차점에서의 전문적인 지도는 프로젝트 기한 내에 필요한 수준으로 세 가지 과제를 완료하는 데 필수적이었습니다.

과제 1	과제 2	과제 3
지진 데이터 수집	PGA 모니터링	과학적 분석
완료	완료	완료

보고서 작성자

Mohammad Tanvir

두잇 주식회사 · DS EQMS — Earthworm 통합 과제 · 2026년3월17일

지도 교수: 권형일 부장님 · 기술 지도: 박병준 주임님

용어 기술 용어 사전

이 용어 사전은 본 보고서 전반에 사용된 주요 기술 용어에 대한 쉬운 정의를 제공합니다. 지진학 또는 소프트웨어 엔지니어링 배경이 없는 경영진 및 비전문가 검토자를 위해 작성되었습니다.

용어	쉬운 설명
Earthworm	전 세계 지진 모니터링 관측소에서 사용되는 오픈 소스 소프트웨어 프레임워크입니다. 공장 조립 라인처럼 작동하여 원시 파형 데이터가 한쪽 끝으로 들어오고 일련의 처리 모듈을 통과하여 깨끗하고 저장되며 접근 가능한 데이터로 나옵니다.
WaveServerV	데이터 서버 역할을 하는 Earthworm 의 구성 요소입니다. 탱크 파일이라는 버퍼에 최근 파형 데이터를 저장하며, Python 스크립트나 SWARM 같은 외부 프로그램이 네트워크를 통해 연결하여 보유한 데이터의 모든 부분을 요청할 수 있습니다.
MiniSEED	지진 파형 데이터 저장을 위한 국제 표준 파일 형식입니다. 전 세계 모든 지진학 소프트웨어가 읽을 수 있습니다. 파형 샘플과 정확히 무엇이, 언제, 어떤 관측소에서 기록되었는지 식별하는 데 필요한 모든 메타데이터를 담는 매우 효율적인 압축 컨테이너라고 생각하면 됩니다.
Steim-2	MiniSEED 파일 내에서 파일 크기를 줄이기 위해 사용되는 압축 알고리즘입니다. 무손실 방식으로 압축 중 데이터 손실이 없습니다. Steim-2 는 지진 정수 데이터를 위해 특별히 설계되어 범용 알고리즘보다 훨씬 더 나은 압축을 달성합니다.
PGA (최대지반가속도)	지진 이벤트 중 주어진 위치에서 지반이 경험하는 최대 가속도로, 1g = 지구의 중력 가속도 ≈ 9.81 m/s^2 인 g 단위로 측정됩니다. PGA 는 지진 공학에서 가장 중요한 측정 중 하나로, 건물이나 구조물이 얼마나 강하게 흔들리는지와 직접 관련이 있습니다.
WAVE_RING	이 배포에서 사용된 Earthworm 내부 공유 메모리 링 버퍼의 이름입니다. 모든 Earthworm 모듈은 이 링에 읽고 쓰는 방식으로 통신합니다. 모든 모듈을 연결하는 중앙 데이터 고속도로입니다. 모든 모듈 구성 파일에서 이름이 정확히 일치해야 합니다.
SWARM	Seismic Wave Analysis and Realtime Monitor. WaveServerV 에 연결하여 화면에 라이브 스크롤 파형 트레이스를 표시하는 Java 기반 데스크톱 애플리케이션입니다. 본 프로젝트에서 데이터가 시스템을 통해 흐르고 있음을 확인하는 독립적인 검증 도구로 사용되었습니다.
ObsPy	지진학을 위해 특별히 설계된 Python 라이브러리입니다. 파형 파일 읽기, 지진 데이터 서버 연결, 지진 신호 처리 도구를 제공합니다. 본 프로젝트에서 WaveServerV 에 연결하고 PGA 계산을 위한 파형 데이터를 검색하는 데 사용되었습니다.
FFT (고속 푸리에 변환)	시간 순서 신호(시간에 따른 진폭)를 주파수 영역 표현(각 주파수에서의 에너지)으로 변환하는 수학 계산입니다. FFT 는 신호에서 어떤 주파수가 가장 두드러진지 보여줍니다. 마치 음악 화음을 개별 음으로 분해하는 것과 같습니다.

용어	쉬운 설명
PSD (전력 스펙트럼 밀도)	신호의 파워(단위 시간당 에너지)가 주파수에 걸쳐 어떻게 분포되는지의 통계적 측정입니다. 본 프로젝트에서 사용된 Welch 방법은 여러 겹치는 FFT 윈도우를 평균 내어 단일 FFT보다 더 부드럽고 신뢰할 수 있는 추정치를 생성합니다.
STFT / 스펙트로그램	단시간 푸리에 변환. 신호가 시간을 통해 진행함에 따라 짧은 겹치는 윈도우에 걸쳐 반복적으로 FFT를 계산하는 기법입니다. 결과인 스펙트로그램은 신호의 주파수 내용이 시간에 따라 어떻게 변하는지 보여주며, 밝은 색상이 높은 에너지를 나타내는 열 지도로 표시됩니다.
DP3 데이터 포트	Q330HRS 레코더에서 사용 가능한 세 개의 데이터 포트 중 세 번째입니다. 각 데이터 포트는 서로 다른 수신자에게 파형 데이터를 스트리밍하도록 구성할 수 있습니다. DP3는 본 프로젝트의 Earthworm 연결을 위해 지정된 포트였습니다.
전송 링	모든 Earthworm 모듈이 데이터를 교환하는 메모리 내 순환 버퍼입니다. 디스크 속도가 아닌 메모리 속도로 작동하여 매우 빠릅니다. 데이터를 공유해야 하는 모든 모듈은 동일한 링 이름을 사용해야 합니다.
감도 계수	원시 디지타이저 정수 카운트를 물리적 단위(이 경우 g)로 변환하는 각 센서와 기록 시스템에 특정한 보정 상수입니다. SM.HSG.HGZ의 경우 값은 2.4e-7 g/count 입니다. 즉, 원시 디지타이저 출력 1카운트는 2.4×10^{-7} g의 지반 가속도에 해당합니다.

부록 부록 — 완전한 Python 소스 코드

이 부록에는 본 프로젝트에서 개발된 세 가지 Python 프로그램의 완전하고 편집되지 않은 소스 코드가 포함되어 있습니다. 이 파일들은 실험 세션 중 테스트 및 검증된 최종 작동 버전을 나타냅니다.

A.1 pga_calculation.py — 실시간 PGA 모니터

```
from obspy.clients.earthworm import Client
from obspy import UTCDateTime
import numpy as np
import time

client = Client("127.0.0.1", 16022)

NET = "SM"
STA = "HSG"
CHA = "HGZ"
LOC = "--"
sensitivity = 2.4e-7

print("=== Checking tank range ===")
try:
    avail = client.get_availability()
    for a in avail:
        print(a)
except Exception as e:
    print("get_availability failed: " + str(e))

print("")
print("=== Starting PGA loop ===")

# Start with 300 seconds back, reduce gradually as tank catches up
offset = 300

while True:
    try:
        t = UTCDateTime() - offset
        st = client.get_waveforms(NET, STA, LOC, CHA, t, t + 1)

        if len(st) == 0 or st[0].stats.npts == 0:
            print(str(UTCDateTime()) + " No data at offset=" + str(offset) + "s,
trying further back...")
            offset += 30
            time.sleep(1)
            continue

        data = st[0].data
        pga_counts = np.max(np.abs(data))
        pga_g = pga_counts * sensitivity
        print(str(UTCDateTime()) + " PGA: " + str(round(pga_g, 8)) + " g  (raw counts:
" + str(pga_counts) + ")  [offset=" + str(offset) + "s]")

        # Gradually reduce offset to get closer to real time
        if offset > 60:
            offset -= 1

    except Exception as e:
```

```
        err = str(e)
        if "FR" in err:
            offset += 30
            print(str(UTCDateTime()) + " Too recent, increasing offset to " +
str(offset) + "s")
        else:
            print(str(UTCDateTime()) + " Error: " + err)

    time.sleep(1)
```

A.2 continuous_mseed.py — 3채널MiniSEED 기록기

```
"""
==============================================================================
  Continuous MSEED Recorder  —  WaveServer / Earthworm
  ────────────────────────────────────────────────────
  Connects to a WaveServer, fetches the last 5 seconds every 5 seconds,
  and saves each trace as a separate .mseed file.

  Files are written to:  ./mseed_data/
  Filename format  :  SM.HSG.HGZ.20260312_040153.mseed
                       (network.station.channel.YYYYMMDD_HHMMSS.mseed)

  Usage:
    python record_continuous_mseed.py

  Dependencies:
    pip install obspy
==============================================================================
"""

from obspy.clients.earthworm import Client
from obspy import UTCDateTime
import time
import os

# ── Configuration ──────────────────────────────────────────────────────────
HOST     = "192.168.0.110"
PORT     = 16022
NETWORK  = "SM"
STATION  = "HSG"
LOCATION = ""
CHANNELS = ["HGZ", "HGN", "HGE"]
INTERVAL = 5          # seconds between each fetch

# Output folder is ALWAYS placed next to this script file — never depends
# on which folder you run the script from
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
OUTDIR     = os.path.join(SCRIPT_DIR, "mseed_data")
# ───────────────────────────────────────────────────────────────────────────

os.makedirs(OUTDIR, exist_ok=True)

print("=" * 60)
print("  Continuous MSEED Recorder")
print(f"  Host    : {HOST}:{PORT}")
print(f"  Station : {NETWORK}.{STATION}  Channels: {', '.join(CHANNELS)}")
print(f"  Output  : {os.path.abspath(OUTDIR)}/")
print(f"  Interval: every {INTERVAL} s")
print("=" * 60)
```

```
print("  Press Ctrl+C to stop.\n")

try:
    client = Client(HOST, PORT)
    print(f"  [✓] Connected to {HOST}:{PORT}\n")
except Exception as e:
    print(f"  [!] Could not connect to {HOST}:{PORT}: {e}\n")
    raise

saved_count  = 0
error_count  = 0

while True:
    t2 = UTCDateTime()
    t1 = t2 - INTERVAL

    for ch in CHANNELS:
        try:
            st = client.get_waveforms(NETWORK, STATION, LOCATION, ch, t1, t2)

            if len(st) == 0:
                print(f"  [~] {ch}  — empty stream, skipping")
                continue

            # Filename uses full timestamp (seconds) so every file is unique
            ts_str   = t1.strftime("%Y%m%d_%H%M%S")
            filename = f"{NETWORK}.{STATION}.{ch}.{ts_str}.mseed"
            filepath = os.path.join(OUTDIR, filename)

            st.write(filepath, format="MSEED")
            saved_count += 1

            n_samp = sum(tr.stats.npts for tr in st)
            print(f"  [✓] Saved: {filename}   ({n_samp} samples)")

        except Exception as e:
            error_count += 1
            print(f"  [!] Error on {ch}: {e}")

    time.sleep(INTERVAL)
```

A.3 analysis.py — MiniSEED 과학적 분석 도구

analysis.py 스크립트는 716줄로, 완전한 MiniSEED 파싱 및 Steim-2 디코딩 파이프라인을 처음부터 구현합니다. ObsPy는 파일 파싱에 사용되지 않습니다. 아래는 정확한 파일 헤더와 함께 함수별 구조 요약입니다.

```
"""
==============================================================================
  MSEED Seismic Data Analysis  —  Pure Python (no ObsPy required)
  ────────────────────────────────────────────────────────────────────────────
  PART 1  :  Parse raw MSEED  +  Steim-2 decode  +  Extract sections
  PART 2  :  Waveform output  (raw / detrended / bandpass-filtered)
  PART 3  :  Frequency analysis  (FFT  /  Welch PSD  /  Spectrogram)
  ────────────────────────────────────────────────────────────────────────────
  Dependencies : pip install numpy scipy matplotlib
  ────────────────────────────────────────────────────────────────────────────
  Usage :
```

```
    python mseed_analysis.py                          <- menu of all .mseed
    python mseed_analysis.py  your_file.mseed         <- single file
==============================================================================
"""

import os, sys, glob, struct, datetime
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from scipy import signal
from scipy.fft import fft, fftfreq

# 핵심 함수:
# parse_header(raw)          — 48바이트 FSDH + 블로켓 체인 디코딩
# decode_steim2(data, n)     — 완전한 Steim-2 비트 수준 압축 해제
# extract_sections(samples)  — 시간 윈도우 추출 (3개 구간)
# process_waveform(samples)  — 평균 제거 + 버터워스 대역 통과 (1-45 Hz)
# frequency_analysis(data)   — FFT + Welch PSD + STFT 스펙트로그램
# plot_results(wf, fa, meta) — 6패널 어노테이션 대시보드 그림
# analyse_file(filepath)     — 마스터 함수: 파싱 → 처리 → 플롯
```

함수	줄수	수행 작업
parse_header(raw)	~50줄	48바이트 FSDH 디코딩 및 블로켓 체인 탐색으로 관측소 메타데이터, 타이밍, 샘플링 속도, 데이터 오프셋 추출
decode_steim2(data)	~70줄	완전한 Steim-2 압축 해제 구현: 64바이트 프레임 읽기, 2비트 프레임 코드 및 4비트 워드 코드로 32비트 워드 해석, 정수 샘플 배열 재구성
extract_sections()	~25줄	디코딩된 샘플 배열을 비교 분석을 위한 세 개의 동일한 시간 윈도우로 분할
process_waveform()	~20줄	DC 오프셋 제거(demean) 및 scipy.signal.filtfilt를 사용한 4극 제로 위상 버터워스 대역 통과 필터 적용
frequency_analysis()	~40줄	처리된 파형에서 FFT 진폭 스펙트럼, Welch PSD, STFT 스펙트로그램 계산
plot_results()	~250줄	어노테이션, 피크 마커, 요약 통계 테이블이 포함된 6패널 대시보드 그림 조합